In [2]:
```
import pandas as pd
from joblib import dump

pitches_train = pd.read_parquet(
    "https://lab.cs307.org/pitches/data/pitches-train.parquet",
)
pitches_test = pd.read_parquet(
    "https://lab.cs307.org/pitches/data/pitches-test.parquet",
)
```

In [3]:
```
# Examine data
pitches_train.head()
pitches_train.info()
pitches_train.describe()
pitches_train["pitch_type"].value_counts()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2868 entries, 0 to 2867
Data columns (total 6 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   pitch_type         2868 non-null   object 
 1   release_speed      2868 non-null   float64
 2   release_spin_rate  2867 non-null   float64
 3   pfx_x              2868 non-null   float64
 4   pfx_z              2868 non-null   float64
 5   stand              2868 non-null   object 
dtypes: float64(4), object(2)
memory usage: 134.6+ KB
```

Out[3]:
```
pitch_type
FF    1488
FS     959
SL     240
SI     181
Name: count, dtype: int64
```

In [4]:
```
# Shape of the train data
n_samples = pitches_train.shape
n_samples
```

Out[4]:
```
(2868, 6)
```

In [5]:
```
target_balance = pitches_train["pitch_type"].value_counts
target_balance
```

Out[5]:
```
<bound method IndexOpsMixin.value_counts of 0       FF
1       FF
2       FF
3       FS
4       FS
        ..
2863    FS
2864    FS
2865    FF
2866    FF
2867    FF
Name: pitch_type, Length: 2868, dtype: object>
```

In [6]:
```
# target balance not normalized
target_balance = pitches_train["pitch_type"].value_counts()
target_balance
```

Out[6]:
```
pitch_type
FF    1488
FS     959
SL     240
SI     181
Name: count, dtype: int64
```

In [7]:
```
# target balance
target_balance = pitches_train["pitch_type"].value_counts(normalize=True)
target_balance
```

Out[7]:
```
pitch_type
FF    0.518828
FS    0.334379
SL    0.083682
SI    0.063110
Name: proportion, dtype: float64
```

In [8]:
```
# velocity stats
velocity_stats = (
    pitches_train.groupby("pitch_type")["release_speed"].agg(["mean", "std"]
)
velocity_stats
```

Out[8]:

pitch_type	mean	std
FF	93.957527	1.700911
FS	85.969552	1.821758
SI	93.310497	1.739876
SL	82.747917	1.864128

In [9]:
```
# spin stats
spin_stats = (
    pitches_train.groupby("pitch_type")["release_spin_rate"].agg(["mean", "s
```

```
)
spin_stats
```

Out[9]:

pitch_type	mean	std
FF	2287.098118	101.983970
FS	1764.038582	177.469770
SI	2189.022099	113.026494
SL	2239.435146	96.452687

In [10]:
```
# Show NA values for test and train
print(pitches_train.isna().sum())
print(pitches_test.isna().sum())
```

```
pitch_type           0
release_speed        0
release_spin_rate    1
pfx_x                0
pfx_z                0
stand                0
dtype: int64
pitch_type            0
release_speed         0
release_spin_rate     0
pfx_x                44
pfx_z                47
stand                 0
dtype: int64
```

In [11]:
```
# create X and y for train
X_train = pitches_train.drop("pitch_type", axis=1)
y_train = pitches_train["pitch_type"]

# create X and y for test
X_test = pitches_test.drop("pitch_type", axis=1)
y_test = pitches_test["pitch_type"]
```

In [ ]:
```

```

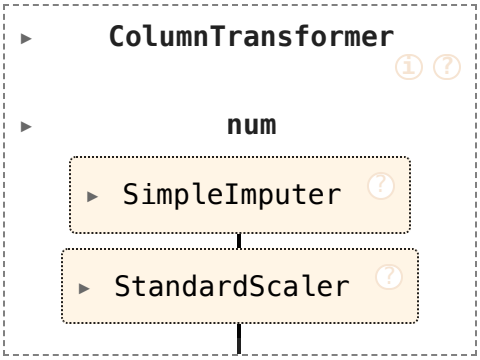
In [12]:
```
# imports
from sklearn.pipeline import Pipeline # composing our model
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

In [13]:
```
# train model on release_speed
numeric_features = ["release_speed"]

# define preproccsing
numeric_transformer = Pipeline(
```

```
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)

column_transformer = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
    ]
)
```

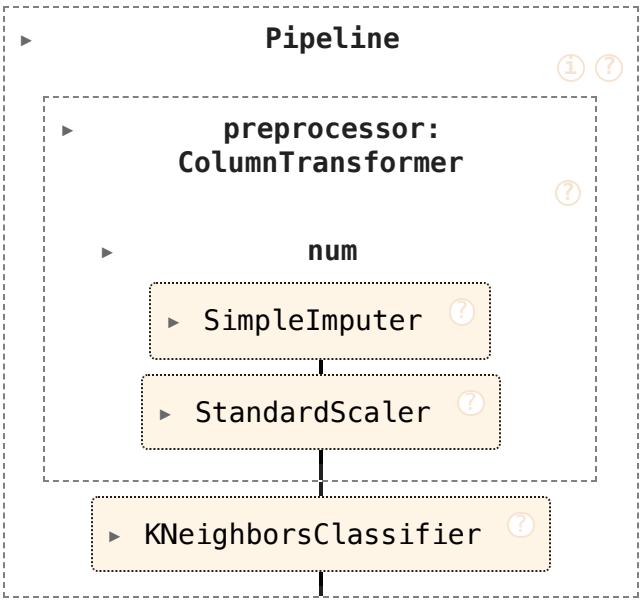
In [14]:
```
column_transformer
```

Out[14]:



In [15]:
```
model = Pipeline(
    steps=[
        ("preprocessor", column_transformer),
        ("classifier", KNeighborsClassifier(n_neighbors=3)),
    ]
)
model
```

Out[15]:



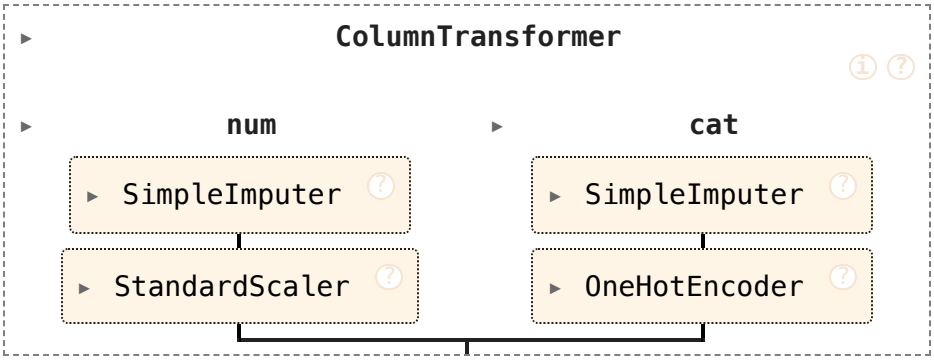
In [16]:
```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

In [17]:
```
accuracy_score(y_test, y_pred)
```

Out[17]:
```
0.867109634551495
```

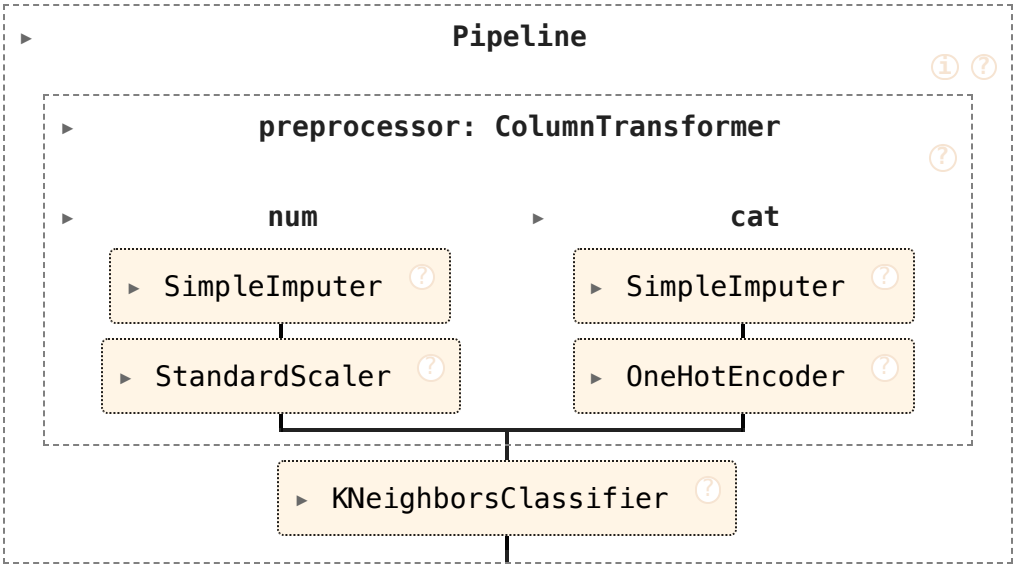
In [18]:
```
# train model on all features
numeric_features = [
    "release_speed",
    "release_spin_rate",
    "pfx_x",
    "pfx_z"]
categorical_features = ["stand"]

# define preproccsing
numeric_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)
categorical_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]
)

column_transformer = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features),
    ]
)
column_transformer
```

Out[18]:



In [19]:
```
model = Pipeline(
    steps=[
        ("preprocessor", column_transformer),
        ("classifier", KNeighborsClassifier(n_neighbors=3)),
    ]
)

model
```

Out[19]:



In [20]:

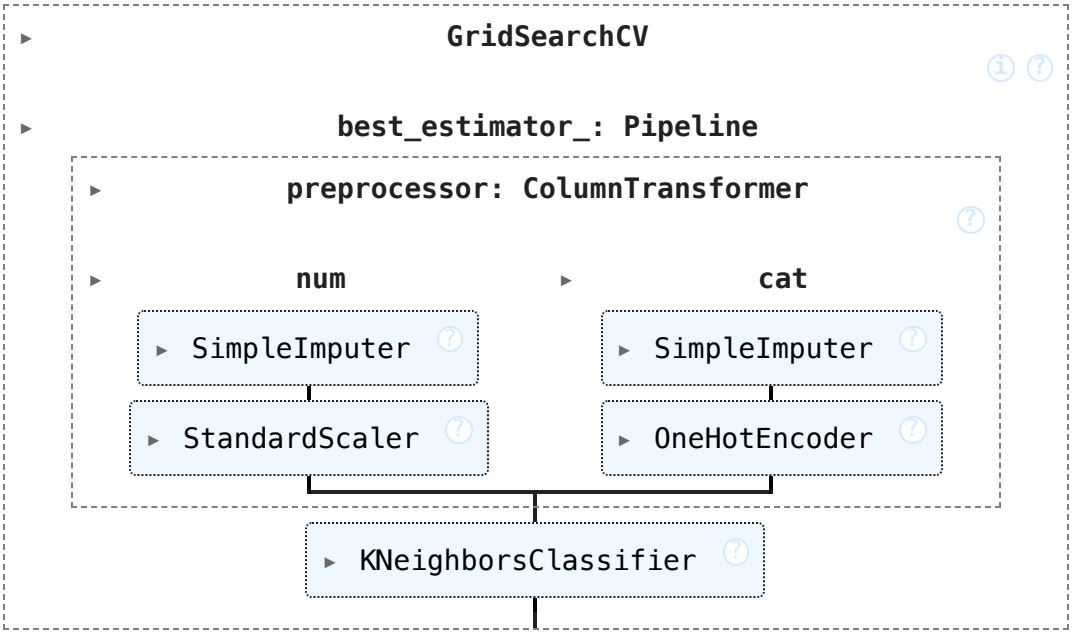
```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Model accuracy: {accuracy:.4f}")
```

```
Model accuracy: 0.9812
```

In [21]:

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'classifier__n_neighbors': [1, 5, 10, 15, 20],
}

model_grid = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
model_grid = model_grid.fit(X_train, y_train)
model_grid
```

Out[21]:



In [22]:

```
model_grid.best_params_["classifier__n_neighbors"]
```

Out[22]:
```
20
```

In [23]:
```
model_grid.cv_results_['mean_test_score']
```

Out[23]:
```
array([0.97733489, 0.97768393, 0.97977513, 0.9804726 , 0.98047382])
```

In [24]:
```
y_pred = model_grid.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Model accuracy after hyperparameter tuning: {accuracy:.4f}")
```

```
Model accuracy after hyperparameter tuning: 0.9862
```

In [25]:
```
# export model for PrairieLearn
dump(model_grid, "pitches.joblib")
```

Out[25]:
```
['pitches.joblib']
```

In [27]:
```
print("You're welcome, Elijah")
print("love, Dulf <3")
```

```
You're welcome, Elijah
love, Dulf <3
```

In [ ]:
```

```